The topology of risk in scientific software

2. Uncertainty and sensitivity analysis

Arnald Puy

Contents

1 Preliminary

```
# PRELIMINARY FUNCTIONS ##################################################
##########################################################################

sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",
                          "cowplot", "readxl", "ggrepel", "tidytext", "here",
                          "tidygraph", "igraph", "foreach", "parallel", "ggraph",
                          "tools", "purrr", "sensobol", "benchmarkme", "softwareRisk",
                          "ggridges"))

# Set seed ---------------------------------------------------------------------

seed <- 123
```

2 The uncertainty and sensitivity analysis

```r
# UNCERTAINTY AND SENSITIVITY ANALYSIS ########################################
################################################################################

# Load the synthetic graph ----------------------------------------------------

data("synthetic_graph")

# Compute all paths -----------------------------------------------------------

out <- all_paths_fun(graph = synthetic_graph)
paths_dt <- data.table(out$paths)

# Define UA / SA settings -----------------------------------------------------

N <- 2^10
order <- "first"

# Run the UA / SA -------------------------------------------------------------

output_ua <- softwareRisk::uncertainty_fun(all_paths_out = out, N = N,
                                           order = order)
```

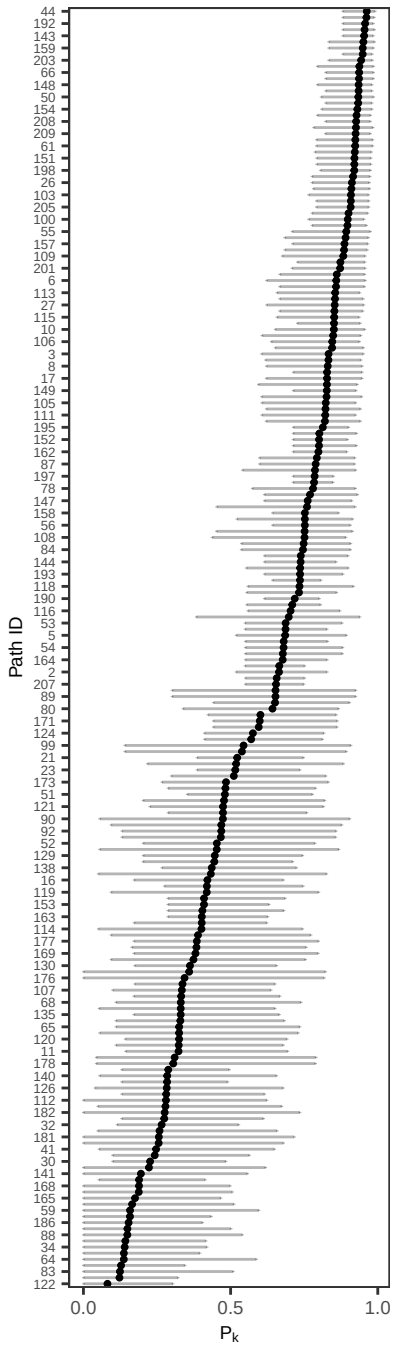
```r
# CREATE THE DATASETS #########################################################

# Create datasets -------------------------------------------------------------

paths_ua_dt <- as.data.table(output_ua[["paths"]])[
  , .(path_id, uncertainty_analysis, gini_index, risk_trend)]

for (nm in c("uncertainty_analysis", "risk_trend", "gini_index")) {

  prefix <- switch(nm, uncertainty_analysis = "P", risk_trend = "risk",
                   gini_index = "gini")

  paths_ua_dt[, c(paste0(prefix, "_median"),
                  paste0(prefix, "_q5"),
                  paste0(prefix, "_q95")
  ) := .(
    vapply(get(nm), median,   numeric(1), na.rm = TRUE),
    vapply(get(nm), quantile, numeric(1), probs = 0.05, na.rm = TRUE),
    vapply(get(nm), quantile, numeric(1), probs = 0.95, na.rm = TRUE)
  )]
}

# Create order to plot --------------------------------------------------------
```

```r
path_order <- paths_ua_dt %>%
  .[order(-P_median)] %>%
  .[, path_id]

# Calculate mean risk per node ------------------------------------------------

nodes_dt <- as.data.table(output_ua[["nodes"]]) %>%
  .[, mean_risk := vapply(uncertainty_analysis, mean, numeric(1), na.rm = TRUE)]

# Extract sensitivity analysis ------------------------------------------------

nodes_dt_unnested <- rbindlist(
  lapply(seq_len(nrow(nodes_dt)), function(i) {
    sa <- as.data.frame(nodes_dt$sensitivity_analysis[[i]]$results)
    sa[["name"]]      <- nodes_dt$name[i]
    sa[["mean_risk"]] <- nodes_dt$mean_risk[i]
    as.data.table(sa)
  })
)
```

3 Figures

```r
# PLOT ERRORBARS ##############################################################

# Plot risk slope -------------------------------------------------------------

paths_ua_dt[, path_id := factor(path_id, levels = rev(path_order))]

# Label every other path to keep the y-axis legible ---------------------------

y_breaks <- levels(paths_ua_dt$path_id)[seq(1, nlevels(paths_ua_dt$path_id), by = 2)]

a <- paths_ua_dt %>%
  ggplot(., aes(P_median, path_id))  +
  geom_point(size = 0.7) +
  geom_errorbar(aes(xmin = P_q5, xmax = P_q95), height = 0.2, alpha = 0.3) +
  labs(y = "Path ID", x = expression(P[k])) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
  scale_y_discrete(breaks = y_breaks) +
  theme(axis.text.y = element_text(size = 5),
        legend.position = c(0.4, 0.87),
        plot.margin = margin(1, 0.1, 1, 1))

a
```

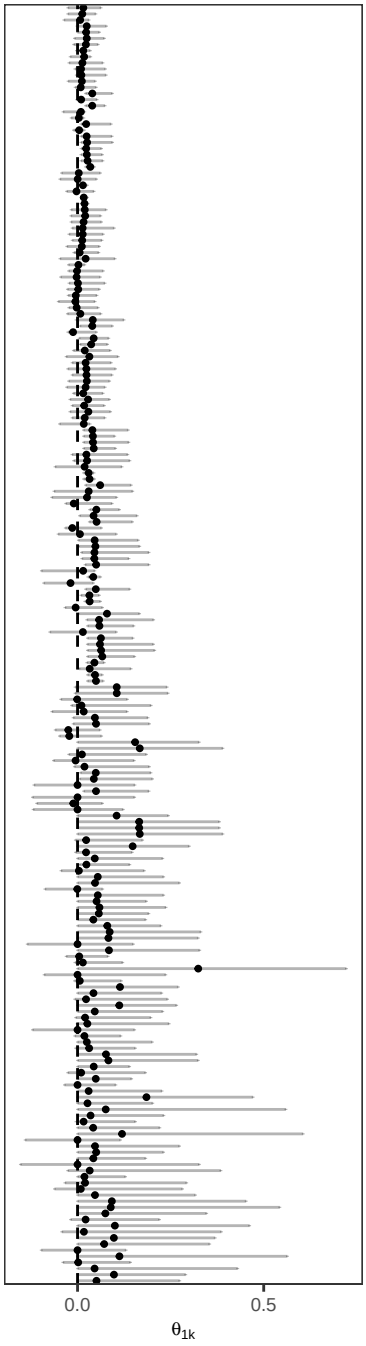
```
## `height` was translated to `width`.
```




```
# Plot risk slope ------------------------------------------------------------

b <- paths_ua_dt %>%
  ggplot(., aes(risk_median, path_id)) +
  geom_point(size = 0.7) +
  geom_vline(xintercept = 0, lty = 2) +
  labs(x = expression(theta[1*k]), y = "") +
  geom_errorbar(aes(xmin = risk_q5, xmax = risk_q95), height = 0.2, alpha = 0.3) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
```

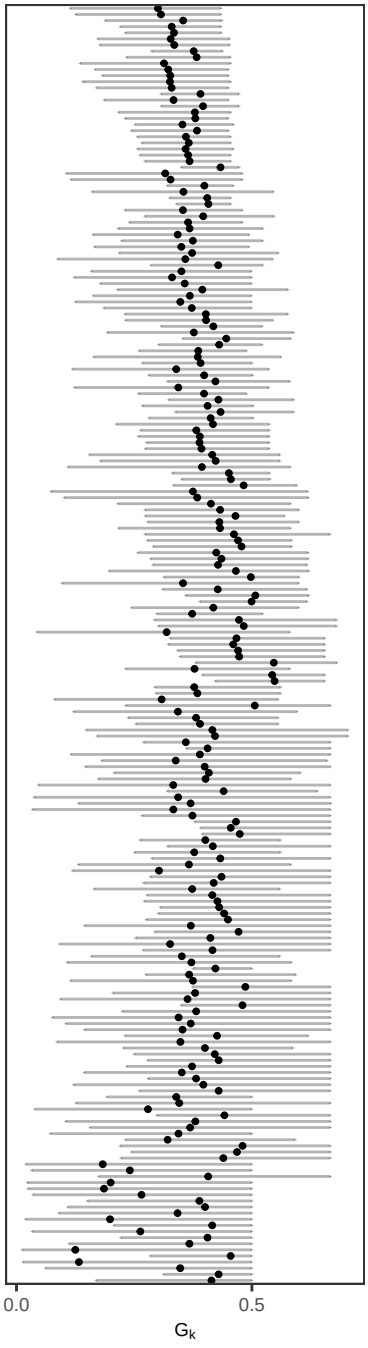
```
  theme(legend.position = "none",
        axis.text.y = element_blank(),
        axis.ticks.y = element_blank(),
        plot.margin = margin(1, 0.1, 1, 0.1))

b
```

```
## `height` was translated to `width`.
```



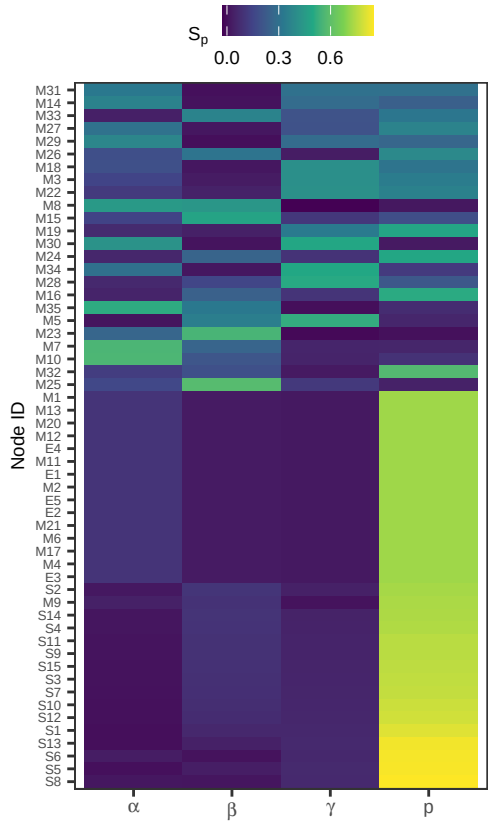
```r
# Plot Gini index ---------------------------------------------------------

c <- paths_ua_dt %>%
  ggplot(., aes(gini_median, path_id)) +
  geom_point(size = 0.7) +
  labs(x = expression(G[k]), y = "") +
  geom_errorbar(aes(xmin = gini_q5, xmax = gini_q95), height = 0.2, alpha = 0.3) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 2)) +
  theme(legend.position = "none",
        axis.text.y = element_blank(),
        axis.ticks.y = element_blank(),
        plot.margin = margin(1, 0.1, 1, 0.1))

c
```

```
## `height` was translated to `width`.
```




```
# PLOT SENSITIVITY ANALYSIS ##########################################################

dt_plot <- nodes_dt_unnested %>%
  .[, .(name, original, sensitivity, parameters)] %>%
  .[sensitivity != "Ti"] %>%
  .[, name := factor(name, levels = unique(name[order(-original)]))]

# Before plotting, convert factor levels to plotmath strings
dt_plot <- dt_plot %>%
  mutate(parameters = recode_factor(parameters,
```

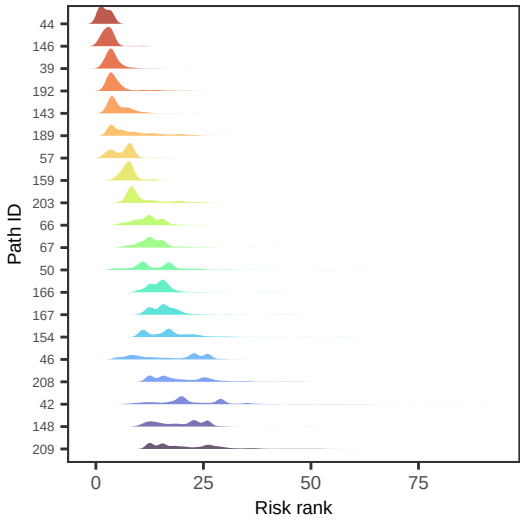
```
                                        "a_raw" = "alpha",
                                        "b_raw" = "beta",
                                        "c_raw" = "gamma",
                                        "p_raw" = "p"))


plot_sa <- ggplot(dt_plot, aes(x = parameters, y = name, fill = original)) +
  geom_tile() +
  scale_fill_viridis_c(name = expression(S[p]), breaks = c(0, 0.3, 0.6)) +
  scale_x_discrete(labels = scales::label_parse()) +
  theme_AP() +
  labs(x = "", y = "Node ID") +
  theme(axis.text.x = element_text(),
        axis.text.y = element_text(size = 5),
        legend.position = "top",
        plot.margin = margin(1, 0.1, 1, 1))

plot_sa
```




```
# RANK DISTRIBUTION PLOT ###########################################################

# Build N x K matrix of path risk draws (rows = draws, cols = paths) -----------

path_ids_chr <- as.character(output_ua$paths$path_id)
```

```r
ua_mat <- do.call(cbind, output_ua$paths$uncertainty_analysis)
colnames(ua_mat) <- path_ids_chr

# For each draw, rank all paths by descending risk (rank 1 = highest risk) -----

rank_mat <- t(apply(ua_mat, 1, function(x) rank(-x, ties.method = "average")))
colnames(rank_mat) <- path_ids_chr

# Select top 20 paths by median rank across draws ------------------------------

med_ranks <- apply(rank_mat, 2, median, na.rm = TRUE)
top_ids   <- path_ids_chr[order(med_ranks)[1:20]]

# Long-format for ggplot -------------------------------------------------------

rank_dt   <- as.data.table(rank_mat[, top_ids, drop = FALSE])
rank_long <- melt(rank_dt, measure.vars = top_ids,
                  variable.name = "path_id", value.name = "rank")
rank_long[, path_id := factor(path_id, levels = rev(top_ids))]

bump_chart_plot <- ggplot(rank_long, aes(x = rank, y = path_id, fill = path_id)) +
  ggridges::geom_density_ridges(alpha = 0.7, scale = 0.9, colour = NA) +
  scale_fill_viridis_d(option = "turbo", end = 0.95) +
  theme_AP() +
  theme(legend.position = "none",
        plot.margin = margin(12, 12, 12, 12),
        axis.text.y = element_text(size = 5)) +
  labs(x = "Risk rank", y = "Path ID")

bump_chart_plot
```

```
## Picking joint bandwidth of 0.92
```



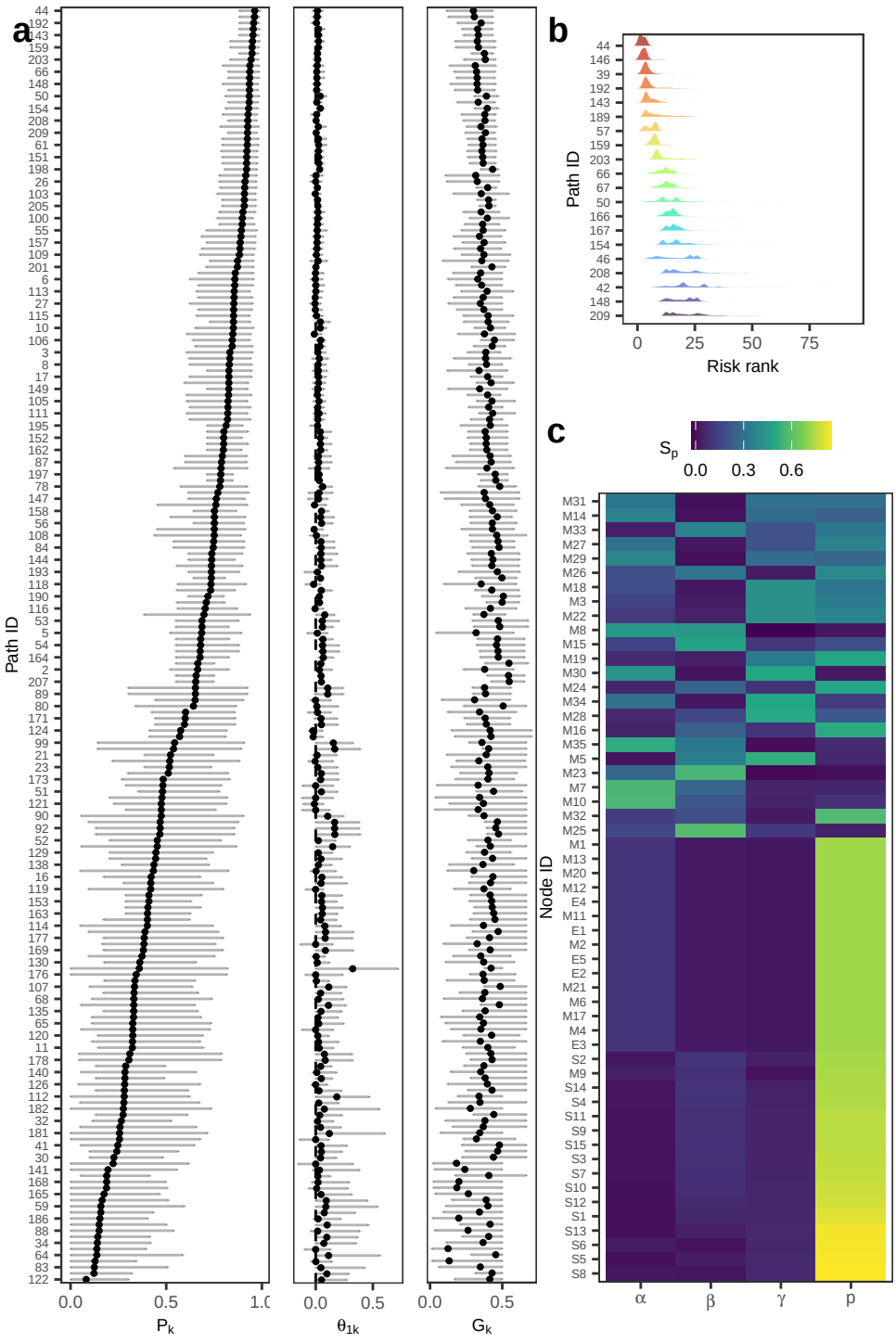
```r
# MERGE UA / SA PLOTS ##########################################################

left <- plot_grid(a, b, c, ncol = 3, rel_widths = c(0.5, 0.25, 0.25), labels = c("a", "", ""))
```

```
## `height` was translated to `width`.
## `height` was translated to `width`.
## `height` was translated to `width`.
```

```r
right <- plot_grid(bump_chart_plot, plot_sa, ncol = 1, rel_heights = c(0.3, 0.7),
                   labels = c("b", "c"))
```

```
## Picking joint bandwidth of 0.92
```

```r
plot_grid(left, right, ncol = 2, rel_widths = c(0.6, 0.4))
```


a
Path ID
Pk
θ1k
Gk
b
Path ID
Risk rank
c
Sp
0.0
0.3
0.6
Node ID
α
β
γ
p

4 Session information

```
# SESSION INFORMATION ############################################################

sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framew
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] tools     parallel  stats     graphics  grDevices utils     datasets
## [8] methods   base
##
## other attached packages:
##  [1] ggridges_0.5.7      softwareRisk_0.2.0  benchmarkme_1.0.8
##  [4] sensobol_1.1.6      ggraph_2.2.2        foreach_1.5.2
##  [7] igraph_2.2.1        tidygraph_1.3.1     here_1.0.2
## [10] tidytext_0.4.3      ggrepel_0.9.6       readxl_1.4.5
## [13] cowplot_1.2.0.9000  scales_1.4.0        openxlsx_4.2.8
## [16] lubridate_1.9.4     forcats_1.0.1       stringr_1.6.0
## [19] dplyr_1.1.4         purrr_1.2.1         readr_2.2.0
## [22] tidyr_1.3.2         tibble_3.3.1        ggplot2_4.0.1.9000
## [25] tidyverse_2.0.0     data.table_1.18.2.1
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1      viridisLite_0.4.2     farver_2.1.2
##  [4] viridis_0.6.5         S7_0.2.1              fastmap_1.2.0
##  [7] tweenr_2.0.3          janeaustenr_1.0.0     digest_0.6.39
## [10] timechange_0.3.0      lifecycle_1.0.5       tokenizers_0.3.0
## [13] magrittr_2.0.4        compiler_4.5.2        rlang_1.1.7
## [16] yaml_2.3.12           knitr_1.51            labeling_0.4.3
## [19] graphlayouts_1.2.2    RColorBrewer_1.1-3    withr_3.0.2
## [22] grid_4.5.2            polyclip_1.10-7       iterators_1.0.14
## [25] MASS_7.3-65           tinytex_0.58          cli_3.6.5
## [28] rmarkdown_2.30        generics_0.1.4        RcppParallel_5.1.11-1
## [31] otel_0.2.0            rstudioapi_0.17.1     httr_1.4.8
## [34] tzdb_0.5.0            cachem_1.1.0          ggforce_0.5.0
```

```
## [37] cellranger_1.1.0      vctrs_0.7.1           Matrix_1.7-4
## [40] hms_1.1.4             ineq_0.2-13           glue_1.8.0
## [43] benchmarkmeData_1.0.4 codetools_0.2-20      rngWELL_0.10-10
## [46] stringi_1.8.7         gtable_0.3.6          randtoolbox_2.0.5
## [49] pillar_1.11.1         htmltools_0.5.9       R6_2.6.1
## [52] zigg_0.0.2            Rdpack_2.6.4          doParallel_1.0.17
## [55] rprojroot_2.1.1       evaluate_1.0.5        lattice_0.22-7
## [58] rbibutils_2.4         SnowballC_0.7.1       Rfast_2.1.5.2
## [61] memoise_2.0.1         Rcpp_1.1.1            zip_2.3.3
## [64] gridExtra_2.3         xfun_0.55             pkgconfig_2.0.3
```

```
## Return the machine CPU ------------------------------------------------------

cat("Machine:     "); print(get_cpu()$model_name)
```

```
## Machine:
## [1] "Apple M1 Max"
```

```
## Return number of true cores -------------------------------------------------

cat("Num cores:   "); print(detectCores(logical = FALSE))
```

```
## Num cores:
## [1] 10
```

```
## Return number of threads ----------------------------------------------------

cat("Num threads: "); print(detectCores(logical = FALSE))
```

```
## Num threads:
## [1] 10
```